

Hurricane Danger!

Day 5 solution walkthrough

Competitive Programming Bootcamp

Statement excerpts:

- ▶ *“city that is closest to the path”*
- ▶ *“travels in a straight line”*

Programming interpretation:

- ▶ The hurricane path is an infinite line through two observed points.
- ▶ Every city's risk is determined by perpendicular distance to that line.
- ▶ Since the denominator is shared, compare only the absolute line-equation numerator.

Textbook connection: CP4 7.2.2 Lines

Core technique: Point-to-line distance comparison

- ▶ The hurricane path is an infinite line through two observed points.
- ▶ Each city has a name and coordinates.
- ▶ Print all cities closest to the hurricane path.

- ▶ Distance is from a point to an infinite line, not to a segment.
- ▶ The denominator of the distance formula is the same for every city in a test case.
- ▶ Comparing only the numerator avoids floating-point equality problems.

- ▶ Convert the two hurricane points into $A*x + B*y + C = 0$.
- ▶ For each city, compute $\text{abs}(A*x + B*y + C)$.
- ▶ If this value is smaller than the best, reset the answer list.
- ▶ If it ties the best, append the city name.
- ▶ Print tied names in input order.

- ▶ $A = y_2 - y_1$, $B = -(x_2 - x_1)$, $C = x_2*y_1 - y_2*x_1$.
- ▶ The full distance divides by $\sqrt{A^2 + B^2}$.
- ▶ Since that denominator is constant, numerator comparison gives the same ranking.

- ▶ The standard line equation represents exactly the hurricane path.
- ▶ Point-to-line distance ranks cities by $\text{abs}(Ax + By + C)$ when the line is fixed.
- ▶ The algorithm keeps exactly all cities with the smallest such value.

Complexity: $O(M)$ time per test case and $O(M)$ memory in the all-tied case.

- ▶ Using integers avoids precision issues when checking ties.
- ▶ Keep city names in input order when distances tie.

Source lines: hurricanedanger.py:45-65

```
import sys

def main():
    input = sys.stdin.buffer.readline

    test_cases = int(input())

    for _ in range(test_cases):
        x1, y1, x2, y2 = map(int, input().split())

        cities = int(input())

        A = y2 - y1
        B = -(x2 - x1)
        C = x2 * y1 - y2 * x1
```

What this chunk does

- ▶ Each case begins with two points on the hurricane path.
- ▶ The line is stored as $A*x + B*y + C = 0$.
- ▶ Those coefficients are reused for every city in the case.

Source lines: hurricanedanger.py:67-83

```
best_distance = None
answer = []

for _ in range(cities):
    data = input().split()

    name = data[0].decode()
    x = int(data[1])
    y = int(data[2])

    distance = abs(A * x + B * y + C)
```

What this chunk does

- ▶ Each city gives a name and integer coordinates.
- ▶ The distance variable is the absolute numerator of point-to-line distance.
- ▶ The omitted denominator is identical for all cities in the test case.

Source lines: hurricanedanger.py:85-98

```
if best_distance is None or distance < best_distance:
    best_distance = distance
    answer = [name]

elif distance == best_distance:
    answer.append(name)

print(" ".join(answer))

main()
```

What this chunk does

- ▶ A smaller distance replaces the current answer list.
- ▶ An equal distance appends the city name in input order.
- ▶ The final line prints all most-dangerous cities separated by spaces.

Solution file: `practice_contests/day_5/hurricanedanger.py`

Problem: <https://open.kattis.com/problems/hurricanedanger>

Use this deck with the source code open beside it. The slides explain the reasoning; the code shows the exact input handling and output formatting.